Guía de estudio – Parcial 1: Algoritmos y Programación 4

Unidad 1: Listas y Recursividad · Unidad 2: Expresiones Regulares · Unidad 3: Conjuntos

Cómo está organizada esta guía: Parte 1 resuelve el parcial anterior (pacientes) con recursividad y trazas de la pila de llamadas. Parte 2 es la guía teórico-práctica de las tres unidades. Parte 3 es un simulador de examen con 6 ejercicios resueltos.

Parte 1 — Resolución del parcial anterior: Sistema de Gestión de Pacientes

1.1 La idea clave: una lista enlazada es recursiva por naturaleza

Una lista enlazada se define así: **lista = vacía, o (nodo + lista más corta)**. Por eso la recursividad encaja perfecto: se resuelve el nodo actual y se "confía" en que la llamada recursiva resuelve el resto.

Hay dos patrones que vas a usar en este parcial:

Patrón	Qué retorna la función recursiva	Se usa en
Reconstruir enlaces al regresar	El nodo que debe quedar "en esa posición"	`agregar`, `filtrar_mayores`
Acumular al regresar	Un valor (contador, suma, booleano)	`contar_criticos`

Regla de oro: como los métodos públicos (`agregar(nombre, edad, urgencia)`) no reciben un nodo, necesitamos un **método auxiliar (helper)** que sí reciba el nodo actual como parámetro. El método público solo hace el "arranque" (`self.cabeza`).

1.2 Código completo comentado

```
class Paciente:
    def __init__(self, nombre, edad, urgencia, atendido=False):
        self.nombre = nombre
        self.edad = edad
        self.urgencia = urgencia      # 1=baja, 5=crítica
        self.atendido = atendido
        self.siguiente = None         # enlace al siguiente nodo
```

```
    def __str__(self):
        estado = "✓" if self.atendido else "○"
        return f"[{estado}] {self.nombre} (edad:{self.edad}) - Urgencia:
{self.urgencia}"


class ListaPacientes:
    def __init__(self):
        self.cabeza = None

    def mostrar(self):
        actual = self.cabeza
        if actual is None:
            print(" Sin pacientes")
            return
        while actual:
            print(f" {actual}")
            actual = actual.siguiente

    # ------------------------------------------------------------------
    # TODO 1: Agregar paciente ordenado por urgencia (mayor a menor)
    # ------------------------------------------------------------------
    def agregar(self, nombre, edad, urgencia):
        nuevo = Paciente(nombre, edad, urgencia)
        # El helper RETORNA la nueva cabeza de la (sub)lista ya con 'nuevo'
insertado.
        self.cabeza = self._insertar(self.cabeza, nuevo)

    def _insertar(self, nodo, nuevo):
        # CASO BASE: llegamos al final (lista vacía) o encontramos la
posición
        # correcta: el nuevo tiene MAYOR urgencia que el nodo actual.
        # Usamos '>' estricto: ante empate, el nuevo va DESPUÉS (respeta
orden de llegada).
        if nodo is None or nuevo.urgencia > nodo.urgencia:
            nuevo.siguiente = nodo   # el nuevo apunta al resto de la lista
            return nuevo             # el nuevo ocupa esta posición

        # PASO RECURSIVO: el nuevo va más adelante. El enlace del nodo actual
        # se "reasigna" con lo que devuelva la llamada sobre el resto.
        nodo.siguiente = self._insertar(nodo.siguiente, nuevo)
        return nodo                  # este nodo se queda en su posición

    # ------------------------------------------------------------------
    # TODO 2: Contar pacientes críticos (urgencia >= 4 y NO atendido)
    # ------------------------------------------------------------------
```

```
    def contar_criticos(self):
        return self._contar_criticos(self.cabeza)

    def _contar_criticos(self, nodo):
        # CASO BASE: lista vacía → no hay críticos.
        if nodo is None:
            return 0
        # ¿El nodo actual cuenta? (1 si es crítico y no atendido, 0 si no)
        cuenta_este = 1 if (nodo.urgencia >= 4 and not nodo.atendido) else 0
        # PASO RECURSIVO: lo mío + lo que cuente el resto.
        return cuenta_este + self._contar_criticos(nodo.siguiente)

    # ------------------------------------------------------------------
    # TODO 3: Filtrar por edad creando una NUEVA ListaPacientes
    # ------------------------------------------------------------------
    def filtrar_mayores(self, edad_minima):
        nueva = ListaPacientes()
        nueva.cabeza = self._filtrar(self.cabeza, edad_minima)
        return nueva

    def _filtrar(self, nodo, edad_minima):
        # CASO BASE: lista vacía → el resultado filtrado también es vacío.
        if nodo is None:
            return None

        # PASO RECURSIVO: primero se filtra TODO el resto; 'resto' es la
cabeza
        # de la lista ya filtrada de los nodos que vienen después.
        resto = self._filtrar(nodo.siguiente, edad_minima)

        if nodo.edad >= edad_minima:
            # Se crea una COPIA (la lista original no se modifica) y se
enlaza al resto.
            copia = Paciente(nodo.nombre, nodo.edad, nodo.urgencia,
nodo.atendido)
            copia.siguiente = resto
            return copia
        # Si no cumple, este nodo se "salta": devolvemos directamente el
resto.
        return resto


# ---------------------------- PRUEBAS ----------------------------
if __name__ == "__main__":
    lista = ListaPacientes()
    lista.agregar("Ana", 30, 2)
```

```
    lista.agregar("Marta", 45, 4)
    lista.agregar("Luis", 70, 5)
    lista.agregar("Pedro", 20, 4)
    lista.agregar("Carla", 60, 3)

    # Marcamos a Pedro como atendido para probar contar_criticos
    p = lista.cabeza
    while p:
        if p.nombre == "Pedro":
            p.atendido = True
        p = p.siguiente

    print("Lista ordenada por urgencia:")
    lista.mostrar()
    print("Críticos sin atender:", lista.contar_criticos())      # 2 (Luis y
Marta)
    print("Mayores de 40:")
    lista.filtrar_mayores(40).mostrar()                          # Luis,
Marta, Carla
```

Salida esperada:

```
Lista ordenada por urgencia:
 [○] Luis (edad:70) - Urgencia:5
 [○] Marta (edad:45) - Urgencia:4
 [✓] Pedro (edad:20) - Urgencia:4
 [○] Carla (edad:60) - Urgencia:3
 [○] Ana (edad:30) - Urgencia:2
Críticos sin atender: 2
Mayores de 40:
 [○] Luis (edad:70) - Urgencia:5
 [○] Marta (edad:45) - Urgencia:4
 [○] Carla (edad:60) - Urgencia:3
```

1.3 TODO 1 — `agregar` (inserción ordenada)

Caso base: `nodo is None` (final de la lista) **o** `nuevo.urgencia > nodo.urgencia` (ya encontramos su lugar). En ambos casos el nuevo se engancha delante de `nodo` y se retorna como nueva "cabeza" de ese tramo.

Paso recursivo: si el nuevo no va aquí, se delega: `nodo.siguiente = _insertar(nodo.siguiente, nuevo)`. La asignación es lo que "cose" la lista al regresar.

Traza — lista actual `Luis(5) → Marta(4) → Pedro(4) → Ana(2)`, se agrega `Carla(3)`:

```
 IDA (se apilan las llamadas)
 _insertar(Luis5,  Carla3)   3 > 5 ? NO  → debo calcular Luis.siguiente
  └─ _insertar(Marta4, Carla3)   3 > 4 ? NO  → debo calcular Marta.siguiente
      └─ _insertar(Pedro4, Carla3)   3 > 4 ? NO  → debo calcular
Pedro.siguiente
          └─ _insertar(Ana2, Carla3)   3 > 2 ? SÍ  → CASO BASE
               Carla.siguiente = Ana ; return Carla

 REGRESO (backtracking, se desapilan)
 Pedro.siguiente = Carla   → return Pedro
 Marta.siguiente = Pedro   → return Marta   (el enlace no cambia)
 Luis.siguiente  = Marta   → return Luis    (el enlace no cambia)
 self.cabeza = Luis

 Resultado: Luis5 → Marta4 → Pedro4 → Carla3 → Ana2
```

Casos especiales que el mismo código ya resuelve: lista vacía (`nodo is None` de inmediato → el nuevo es cabeza); urgencia mayor que todos (se inserta en la primera llamada); urgencia menor que todos (llega a `None` y queda al final).

Error común: olvidar el `return nodo` al final del paso recursivo o no asignar `nodo.siguiente = ...`. Sin la asignación, el nuevo nodo se crea pero nunca queda enlazado.

Complejidad: tiempo O(n) (peor caso recorre toda la lista); espacio O(n) por la pila de recursión.

1.4 TODO 2 — `contar_criticos` (acumular al regresar)

Caso base: `nodo is None` → `0` (una lista vacía no tiene críticos).

Paso recursivo: `(1 si este nodo es crítico y no atendido, si no 0) + contar(resto)`.

Traza — lista `Luis5○ → Marta4○ → Pedro4✓ → Carla3○ → Ana2○`:

```
IDA
contar(Luis)  = [1] + contar(Marta)
 └─ contar(Marta) = [1] + contar(Pedro)
     └─ contar(Pedro) = [0] + contar(Carla)     (Pedro ya fue atendido)
         └─ contar(Carla) = [0] + contar(Ana)   (urgencia 3 < 4)
             └─ contar(Ana) = [0] + contar(None)
                 └─ contar(None) = 0            ← CASO BASE

REGRESO (las respuestas se acumulan)
contar(Ana)   = 0 + 0 = 0
contar(Carla) = 0 + 0 = 0
contar(Pedro) = 0 + 0 = 0
contar(Marta) = 1 + 0 = 1
contar(Luis)  = 1 + 1 = 2   ← respuesta final
```

Error común: usar `or` en lugar de `and` (contaría atendidos de urgencia alta o cualquier no atendido), o escribir `nodo.atendido == False` mal combinado. La condición correcta es `urgencia >= 4 and not atendido`.

Complejidad: O(n) tiempo, O(n) espacio (pila). Es recursión **lineal, no de cola** en el sentido estricto: después de la llamada todavía hay que sumar `cuenta_este`.

1.5 TODO 3 — `filtrar_mayores` (nueva lista por backtracking)

Caso base: `nodo is None` → `None` (lista filtrada vacía).

Paso recursivo: filtrar primero el resto (`resto`). Al regresar, si el nodo cumple, se **copia** y se enlaza a `resto`; si no cumple, se retorna `resto` tal cual (el nodo se omite).

Por qué se copia: el enunciado pide una **NUEVA** lista. Si se reutilizaran los nodos originales, ambas listas compartirían nodos y modificar una dañaría la otra.

Traza — filtrar `edad >= 40` sobre `Luis70 → Marta45 → Pedro20 → Carla60 → Ana30`:

```
IDA
_filtrar(Luis70)
 └─ _filtrar(Marta45)
     └─ _filtrar(Pedro20)
         └─ _filtrar(Carla60)
             └─ _filtrar(Ana30)
                 └─ _filtrar(None) → None   ← CASO BASE

REGRESO (aquí se construye la lista nueva, de atrás hacia adelante)
Ana30   : 30 < 40  → NO cumple → return resto = None
Carla60 : 60 ≥ 40  → copia Carla' ; Carla'.sig = None  → return Carla'
Pedro20 : 20 < 40  → NO cumple → return resto = Carla'
Marta45 : 45 ≥ 40  → copia Marta' ; Marta'.sig = Carla' → return Marta'
Luis70  : 70 ≥ 40  → copia Luis'  ; Luis'.sig  = Marta' → return Luis'

nueva.cabeza = Luis'  →  Luis' → Marta' → Carla'
```

Observa que el **orden original se conserva** (no hace falta invertir nada) porque la lista se arma al regresar, empezando por el último nodo.

Error común: modificar la lista original (`nodo.siguiente = ...`) en lugar de crear copias, o llamar a `self.agregar` dentro del filtro (reordenaría y haría O(n²)).

Complejidad: O(n) tiempo y O(n) espacio de pila (más O(k) nodos nuevos).

1.6 Resumen rápido de la Parte 1

Método	Caso base	Paso recursivo	Tiempo	Pila
`agregar`	`nodo is None` o `nuevo.urgencia > nodo.urgencia`	`nodo.siguiente = _insertar(nodo.siguiente, nuevo)`	O(n)	O(n)
`contar_criticos`	`nodo is None` → 0	`cuenta_este + contar(resto)`	O(n)	O(n)
`filtrar_mayores`	`nodo is None` → `None`	filtrar resto; copiar si cumple, omitir si no	O(n)	O(n)

Parte 2 — Guía teórico-práctica por unidad

A. Unidad 1: Listas enlazadas y recursividad

A.1 Diferencias estructurales entre los tres tipos de lista

Característica	Simplemente enlazada	Circular simplemente enlazada	Doblemente enlazada
Enlaces por nodo	1 (`siguiente`)	1 (`siguiente`)	2 (`anterior`, `siguiente`)
Último nodo apunta a	`None`	Al primer nodo	`None` (y el primero tiene `anterior = None`)
Condición de parada al recorrer	`actual is None`	`actual is cabeza` (volvió al inicio)	`actual is None`
Recorrer hacia atrás	No	No	Sí
Eliminar un nodo conocido	O(n): hay que buscar el previo	O(n)	O(1) si ya tienes el nodo
Uso típico	Pilas, colas, listas simples	Turnos rotativos (round-robin)	Historial, deshacer/rehacer

Lista simplemente enlazada

```
class Nodo:
    def __init__(self, dato):
        self.dato = dato          # información almacenada
        self.siguiente = None     # referencia al siguiente nodo


class ListaSimple:
    def __init__(self):
        self.cabeza = None        # lista vacía

    def agregar_final(self, dato):
        nuevo = Nodo(dato)
        if self.cabeza is None:   # caso especial: lista vacía
            self.cabeza = nuevo
            return
```

```
        actual = self.cabeza
        while actual.siguiente is not None:   # avanzar hasta el último nodo
            actual = actual.siguiente
        actual.siguiente = nuevo

    def mostrar(self):
        elementos = []
        actual = self.cabeza
        while actual is not None:             # parada: se acabó la lista
            elementos.append(str(actual.dato))
            actual = actual.siguiente
        print(' -> '.join(elementos) + ' -> None')


l = ListaSimple()
for x in (10, 20, 30):
    l.agregar_final(x)
l.mostrar()      # 10 -> 20 -> 30 -> None
```

Lista circular simplemente enlazada

Truco muy útil: se guarda una referencia al **último** nodo. Así `ultimo.siguiente` es la cabeza, y agregar al inicio o al final es O(1).

```
class ListaCircular:
    def __init__(self):
        self.ultimo = None        # el último nodo; ultimo.siguiente es la
cabeza

    def agregar_final(self, dato):
        nuevo = Nodo(dato)
        if self.ultimo is None:               # primer nodo: se apunta a sí
mismo
            nuevo.siguiente = nuevo
        else:
            nuevo.siguiente = self.ultimo.siguiente   # el nuevo apunta a la
cabeza
            self.ultimo.siguiente = nuevo             # el antiguo último
apunta al nuevo
        self.ultimo = nuevo                           # el nuevo es ahora el
último

    def mostrar(self):
        if self.ultimo is None:
            print('Lista vacía')
            return
```

```
        cabeza = self.ultimo.siguiente
        actual = cabeza
        elementos = []
        while True:                          # estilo do-while: se ejecuta
al menos una vez
            elementos.append(str(actual.dato))
            actual = actual.siguiente
            if actual is cabeza:             # dimos la vuelta completa
                break
        print(' -> '.join(elementos) + f' -> (vuelve a {elementos[0]})')

    def eliminar(self, dato):
        if self.ultimo is None:
            return False
        previo = self.ultimo                 # el previo de la cabeza es el
último
        actual = self.ultimo.siguiente
        while True:
            if actual.dato == dato:
                if actual is previo:         # era el único nodo
                    self.ultimo = None
                else:
                    previo.siguiente = actual.siguiente   # se salta el nodo
                    if actual is self.ultimo: # si borramos el último,
actualizar
                        self.ultimo = previo
                return True
            previo = actual
            actual = actual.siguiente
            if previo is self.ultimo:        # ya revisamos todos los nodos
                break
        return False


c = ListaCircular()
for x in ('A', 'B', 'C'):
    c.agregar_final(x)
c.mostrar()      # A -> B -> C -> (vuelve a A)
c.eliminar('B')
c.mostrar()      # A -> C -> (vuelve a A)
```

Error común: recorrer una lista circular con `while actual is not None` → ciclo infinito, porque nunca hay `None`.

Lista doblemente enlazada

```
class NodoDoble:
    def __init__(self, dato):
        self.dato = dato
        self.anterior = None
        self.siguiente = None


class ListaDoble:
    def __init__(self):
        self.cabeza = None
        self.cola = None          # referencia al último: agregar al final es
O(1)

    def agregar_final(self, dato):
        nuevo = NodoDoble(dato)
        if self.cola is None:                 # lista vacía
            self.cabeza = self.cola = nuevo
        else:
            nuevo.anterior = self.cola        # enlace hacia atrás
            self.cola.siguiente = nuevo       # enlace hacia adelante
            self.cola = nuevo

    def eliminar(self, dato):
        actual = self.cabeza
        while actual is not None:
            if actual.dato == dato:
                # Reconectar el vecino de la izquierda
                if actual.anterior is not None:
                    actual.anterior.siguiente = actual.siguiente
                else:                         # era la cabeza
                    self.cabeza = actual.siguiente
                # Reconectar el vecino de la derecha
                if actual.siguiente is not None:
                    actual.siguiente.anterior = actual.anterior
                else:                         # era la cola
                    self.cola = actual.anterior
                return True
            actual = actual.siguiente
        return False

    def mostrar_adelante(self):
        r, a = [], self.cabeza
        while a:
            r.append(str(a.dato)); a = a.siguiente
```

```
        print(' <-> '.join(r))

    def mostrar_atras(self):
        r, a = [], self.cola
        while a:
            r.append(str(a.dato)); a = a.anterior
        print(' <-> '.join(r))


d = ListaDoble()
for x in (1, 2, 3, 4):
    d.agregar_final(x)
d.eliminar(3)
d.mostrar_adelante()   # 1 <-> 2 <-> 4
d.mostrar_atras()      # 4 <-> 2 <-> 1
```

Error común: al eliminar en una doble, actualizar solo `siguiente` y olvidar `anterior` (o no actualizar `cabeza` / `cola` en los extremos).

A.2 Tipos de recursividad y análisis de complejidad

Tipo	Descripción	Ejemplo
Lineal	Una sola llamada recursiva por ejecución	`factorial(n) = n * factorial(n-1)`
De cola (tail)	La llamada recursiva es lo ÚLTIMO que se hace (usa acumulador)	`suma(n, acc) = suma(n-1, acc+n)`
Múltiple / binaria	Dos o más llamadas por ejecución	`fib(n) = fib(n-1) + fib(n-2)`
Indirecta (mutua)	A llama a B y B llama a A	`es_par(n)` / `es_impar(n)`
Anidada	El argumento de la llamada es otra llamada	Función de Ackermann

```
def factorial(n):                # lineal
    if n <= 1: return 1          # caso base
    return n * factorial(n - 1)

def suma_cola(n, acc=0):         # de cola
    if n == 0: return acc
    return suma_cola(n - 1, acc + n)

def fib(n):                      # múltiple
    if n < 2: return n
    return fib(n - 1) + fib(n - 2)

def es_par(n):                   # indirecta
    return True if n == 0 else es_impar(n - 1)
def es_impar(n):
    return False if n == 0 else es_par(n - 1)
```

Análisis de complejidad con relaciones de recurrencia:

Algoritmo	Recurrencia	Resultado
Recorrido recursivo de lista de n nodos	$T(n) = T(n-1) + O(1)$	$O(n)$ tiempo, $O(n)$ pila
`factorial`	$T(n) = T(n-1) + O(1)$	$O(n)$
`fib` ingenuo	$T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$ (exactamente $O(\varphi^n)$)
Búsqueda binaria recursiva	$T(n) = T(n/2) + O(1)$	$O(\log n)$
Inversión recursiva de lista	$T(n) = T(n-1) + O(1)$	$O(n)$ tiempo, $O(n)$ pila

Método de sustitución (idea): $T(n) = T(n-1) + 1 = T(n-2) + 2 = \ldots = T(0) + n = O(n)$.

Detalle de Python: el límite de recursión por defecto es ~1000 (`sys.getrecursionlimit()`), así que una lista de 5000 nodos recorrida recursivamente lanza `RecursionError` . Python **no** optimiza la recursión de cola.

A.3 Patrones recursivos en listas: directa vs. helper

Recursividad directa: una función libre que recibe el nodo como parámetro; el usuario debe pasar `lista.cabeza` .

Método auxiliar (helper): un método público sin parámetros de nodo que arranca la recursión con `self.cabeza` y delega a un método privado `_algo(nodo, ...)`. Es el estilo que se espera en el parcial porque el usuario de la clase no debería manejar nodos.

```
# ---------- Estilo DIRECTO (función libre) ----------
def buscar_directo(nodo, dato):
    if nodo is None:            # caso base 1: no está
        return False
    if nodo.dato == dato:       # caso base 2: lo encontramos
        return True
    return buscar_directo(nodo.siguiente, dato)   # paso recursivo

# ---------- Estilo HELPER (método de la clase) ----------
class ListaRec(ListaSimple):
    # ---- BÚSQUEDA ----
    def buscar(self, dato):                       # público
        return self._buscar(self.cabeza, dato)    # arranca la recursión

    def _buscar(self, nodo, dato):                # helper
        if nodo is None:
            return False
        if nodo.dato == dato:
            return True
        return self._buscar(nodo.siguiente, dato)

    # ---- ELIMINACIÓN (primera ocurrencia) ----
    def eliminar(self, dato):
        self.cabeza = self._eliminar(self.cabeza, dato)

    def _eliminar(self, nodo, dato):
        if nodo is None:                 # caso base: no estaba
            return None
        if nodo.dato == dato:            # caso base: lo saltamos
devolviendo su siguiente
            return nodo.siguiente
        nodo.siguiente = self._eliminar(nodo.siguiente, dato)   # re-enlazar
al volver
        return nodo

    # ---- INVERSIÓN (con retorno de la nueva cabeza) ----
    def invertir(self):
        self.cabeza = self._invertir(self.cabeza)

    def _invertir(self, nodo):
        if nodo is None or nodo.siguiente is None:   # vacía o último nodo
            return nodo                              # el último será la
```

```
nueva cabeza
        nueva_cabeza = self._invertir(nodo.siguiente)  # invierte el resto
        nodo.siguiente.siguiente = nodo   # el siguiente ahora apunta hacia
atrás a mí
        nodo.siguiente = None             # yo quedo como final provisional
        return nueva_cabeza               # se propaga la misma cabeza hasta
arriba

    # ---- INVERSIÓN versión de COLA (con acumulador 'previo') ----
    def _invertir_cola(self, actual, previo=None):
        if actual is None:
            return previo
        sig = actual.siguiente
        actual.siguiente = previo         # se voltea el enlace
        return self._invertir_cola(sig, actual)
```

Traza de `_invertir` sobre `1 → 2 → 3`:

```
 IDA
 _invertir(1) → _invertir(2) → _invertir(3)   3 es el último: return 3 (CASO
BASE)

 REGRESO
 En _invertir(2): nueva_cabeza = 3
    nodo.siguiente.siguiente = nodo   →  3.siguiente = 2      (3 → 2)
    nodo.siguiente = None             →  2.siguiente = None   (3 → 2 → None)
    return 3
 En _invertir(1): nueva_cabeza = 3
    nodo.siguiente.siguiente = nodo   →  2.siguiente = 1      (3 → 2 → 1)
    nodo.siguiente = None             →  1.siguiente = None   (3 → 2 → 1 →
None)
    return 3   → self.cabeza = 3
```

Cuándo usar cada patrón:

Operación	Qué retorna el helper	Por qué
Buscar / contar / sumar	Un valor (bool, int)	Solo se consulta, no se modifica
Insertar / eliminar / filtrar	El nodo que queda en esa posición	Se necesita re-enlazar al volver, y la cabeza puede cambiar
Invertir	La nueva cabeza (último nodo original)	La cabeza cambia por completo

---

B. Unidad 2: Expresiones regulares

B.1 Representación, evaluación y conversión

Una expresión regular (ER) describe un **lenguaje** (conjunto de cadenas). Se puede escribir en tres representaciones equivalentes:

Representación	Operadores	Ejemplo (cadenas que terminan en `ab`)
Formal (teórica)	unión `+` o `∪` · concatenación (juntar) · estrella `*` · `ε` (cadena vacía) · `∅`	`(a+b)*ab`
Extendida (azúcar sintáctica)	además `+` (una o más), `?`, `{n,m}`, clases `[a-z]`	`[ab]*ab`
Python (`re`)	`|`, `*`, `+`, `?`, `{n,m}`, `[]`, `^`, `$`, `\d`, `\w`, grupos `()`	`^[ab]*ab$`

Conversiones de azúcar sintáctica a forma formal:

Extendida	Formal equivalente
`r+`	`r r*`
`r?`	`r + ε`
`r{3}`	`r r r`
`r{2,3}`	`r r (r + ε)`
`[abc]`	`a + b + c`
`[a-c]`	`a + b + c`

Identidades para simplificar expresiones:

Identidad	Nombre / idea
`r + r = r`	Idempotencia de la unión
`r + ∅ = r` · `r ε = r` · `r ∅ = ∅`	Elementos neutro y absorbente
`(r*)* = r*`	Estrella doble sobra
`ε + r r* = r*`	Definición de estrella

Identidad	Nombre / idea
`(a+b)* = (a*b*)* = (a*+b*)*`	Distintas formas del mismo lenguaje (todas las cadenas sobre {a,b})
`ab + ac = a(b + c)`	Factorización (distributiva)
`r*r* = r*`	La estrella absorbe a sí misma

Cómo evaluar si una cadena pertenece a la ER: descomponer la cadena según la estructura de la expresión.

Ejemplo con `a*b+` (cero o más `a`, seguidas de una o más `b`):

Cadena	Descomposición	¿Pertenece?
`aabb`	`aa` · `bb`	Sí
`b`	`ε` · `b`	Sí
`aba`	`a` · `b` · sobra `a`	No
`aa`	falta al menos una `b`	No

Las ER, los autómatas finitos deterministas (AFD) y los no deterministas (AFN) describen **exactamente la misma clase de lenguajes** (los regulares). Por eso un motor de regex puede construir internamente un autómata a partir de la ER.

Verificar equivalencia con Python (como prueba, no como demostración):

```
import re
from itertools import product

def equivalentes(r1, r2, alfabeto='ab', largo_max=6):
    '''Prueba todas las cadenas hasta largo_max y compara si ambas ER las
aceptan.'''
    p1, p2 = re.compile(r1), re.compile(r2)
    for n in range(largo_max + 1):
        for tupla in product(alfabeto, repeat=n):
            s = ''.join(tupla)
            if bool(p1.fullmatch(s)) != bool(p2.fullmatch(s)):
                return False, s           # cadena contraejemplo
    return True, None

print(equivalentes(r'(a|b)*', r'(a*b*)*'))      # (True, None)
print(equivalentes(r'a+', r'aa*'))              # (True, None)
print(equivalentes(r'(a|b)*a', r'(a|b)*b'))     # (False, 'a')
```

Recordatorio de funciones de `re`:

Función	Qué hace
`re.fullmatch(p, s)`	La cadena COMPLETA debe cumplir el patrón (lo más seguro para validar)
`re.match(p, s)`	Coincide solo desde el INICIO de la cadena
`re.search(p, s)`	Primera coincidencia en cualquier posición
`re.findall(p, s)`	Lista con todas las coincidencias
`re.finditer(p, s)`	Iterador de objetos match (con grupos y posiciones)
`re.sub(p, reemplazo, s)`	Reemplaza coincidencias

B.2 Cinco ejemplos prácticos con `re`

Ejemplo 1 — Validar correo electrónico

```
import re

patron_correo = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$'

for c in ('ana.perez@udea.edu.co', 'juan@gmail.com', 'ana@@x.com', 'ana@x', '@x.com'):
    print(c, '->', bool(re.fullmatch(patron_correo, c)))
# True, True, False, False, False
```

Desglose carácter por carácter:

Fragmento	Significado
`^`	Inicio de la cadena
`[A-Za-z0-9._%+-]+`	Parte local: una o más letras, dígitos o `. _ % + -` (el `-` al final es literal)
`@`	Arroba literal
`[A-Za-z0-9-]+`	Primer segmento del dominio (`udea`)
`(?:\.[A-Za-z0-9-]+)*`	Cero o más subdominios `.algo` (grupo **sin captura** `?:`); `\.` es un punto literal

Fragmento	Significado
`\.`	Punto antes de la extensión
`[A-Za-z]{2,}`	Extensión de 2 o más letras (`co`, `com`)
`$`	Fin de la cadena

Ejemplo 2 — Contraseña segura con lookaheads

Requisitos: mínimo 8 caracteres, una minúscula, una mayúscula, un dígito y un símbolo.

```
patron_pass = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+\-=]).{8,}$'

for p in ('Abcd123!', 'abcd1234', 'ABCD123!', 'Ab1!', 'Clave#2026'):
    print(p, '->', bool(re.fullmatch(patron_pass, p)))
# True, False, False, False, True
```

Desglose:

Fragmento	Significado
`^`	Inicio
`(?=.*[a-z])`	**Lookahead positivo**: desde el inicio, en algún punto hay una minúscula. NO consume caracteres: solo verifica
`(?=.*[A-Z])`	Igual, exige una mayúscula
`(?=.*\d)`	Exige un dígito
`(?=.*[!@#$%^&*()_+\-=])`	Exige un símbolo (`\-` escapa el guion)
`.{8,}`	Ahora sí consume: 8 o más caracteres cualesquiera
`$`	Fin

Idea clave: como los lookaheads no consumen, todos se evalúan desde la **misma posición inicial**, y funcionan como un "AND" de condiciones.

Ejemplo 3 — Extraer números telefónicos (celulares de Colombia)

```
texto = 'Llámame al 300 123 4567 o al +57 3151234567, el fijo es 604-
1234567.'
patron_cel = r'(?:\+57[\s-]?)?\b3\d{2}[\s-]?\d{3}[\s-]?\d{4}\b'

print(re.findall(patron_cel, texto))
# ['300 123 4567', '+57 3151234567']   (el fijo 604-... no empieza por 3)
```

Fragmento	Significado
`(?:\+57[\s-]?)?`	Prefijo opcional `+57` seguido de espacio o guion opcional
`\b`	Frontera de palabra (evita empezar en medio de otro número)
`3\d{2}`	Un `3` y dos dígitos (los celulares colombianos empiezan por 3)
`[\s-]?`	Separador opcional (espacio o guion)
`\d{3}` · `\d{4}`	Grupos de 3 y 4 dígitos
`\b`	Frontera final

Ejemplo 4 — Extraer y reordenar fechas con grupos

```
texto = 'Entrega el 05/11/2026 y revisión el 31/12/2026. Fecha mala:
45/13/2026.'
patron_fecha = r'\b(0[1-9]|[12]\d|3[01])/(0[1-9]|1[0-2])/(\d{4})\b'

for m in re.finditer(patron_fecha, texto):
    dia, mes, anio = m.groups()
    print(f'Día {dia}, mes {mes}, año {anio}')   # solo las dos fechas
válidas

# Convertir DD/MM/AAAA a AAAA-MM-DD usando referencias a grupos
print(re.sub(patron_fecha, r'\3-\2-\1', texto))
```

Fragmento	Significado
`(0[1-9]|[12]\d|3[01])`	Grupo 1, día válido: 01–09, 10–29 o 30–31
`/`	Barra literal

Fragmento	Significado
`(0[1-9]|1[0-2])`	Grupo 2, mes válido: 01–09 o 10–12
`(\d{4})`	Grupo 3, año de 4 dígitos
`\3-\2-\1`	En el reemplazo: año, mes, día (referencias a los grupos)

Ejemplo 5 — Reemplazar y formatear con `re.sub`

```
# a) Colapsar espacios múltiples
print(re.sub(r'\s+', ' ', '  Hola    mundo   ').strip())          # 'Hola
mundo'

# b) Formatear un celular: 3001234567 -> (300) 123-4567
print(re.sub(r'(\d{3})(\d{3})(\d{4})', r'(\1) \2-\3', '3001234567'))

# c) Capitalizar nombres usando una función como reemplazo
print(re.sub(r'\b(\w)(\w*)\b', lambda m: m.group(1).upper() +
m.group(2).lower(), 'jUAN pérez'))
# 'Juan Pérez'

# d) Enmascarar una tarjeta dejando visibles los últimos 4 dígitos
print(re.sub(r'\d(?=\d{4})', '*', '1234567890123456'))         #
'************3456'
```

Patrón	Desglose
`\s+`	Uno o más espacios, tabs o saltos de línea
`(\d{3})(\d{3})(\d{4})`	Tres grupos de captura: `\1`, `\2`, `\3` en el reemplazo
`\b(\w)(\w*)\b`	Una palabra: primera letra (grupo 1) y el resto (grupo 2)
`\d(?=\d{4})`	Un dígito **seguido de** 4 dígitos más (lookahead): todos menos los últimos 4

Errores comunes en regex: olvidar `r''` (cadena cruda) y tener que duplicar las barras; usar `match` creyendo que valida toda la cadena (usa `fullmatch` o `^...$`); olvidar escapar el punto (`.` sin escapar significa "cualquier carácter"); cuantificadores codiciosos (`.*`) que se comen más de lo esperado, solución: `.*?`.

C. Unidad 3: Teoría e implementación de conjuntos

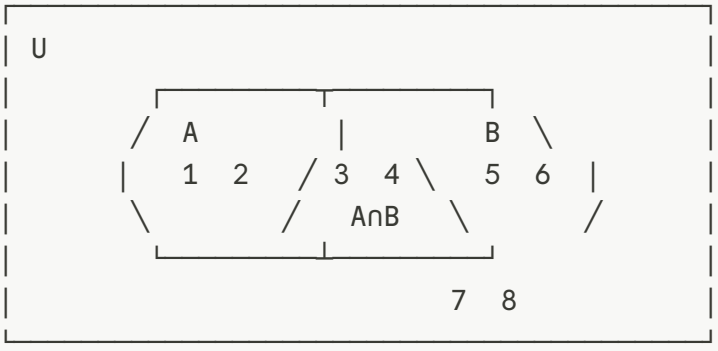
C.1 Teoría y operaciones

Un **conjunto** es una colección de elementos **distintos** y **sin orden**. Para hablar de complemento hace falta un **universo U** (el conjunto de todos los elementos posibles).

Datos para todos los ejemplos: **U = {1,2,3,4,5,6,7,8}**, **A = {1,2,3,4}**, **B = {3,4,5,6}**.

Operación	Símbolo	Definición	Resultado del ejemplo	Región en el diagrama de Venn
Unión	A ∪ B	Elementos que están en A **o** en B (o en ambos)	{1,2,3,4,5,6}	Se sombrean los DOS círculos completos
Intersección	A ∩ B	Elementos que están en A **y** en B	{3,4}	Solo la zona donde los círculos se traslapan (la "almendra")
Diferencia	A \ B	Elementos de A que **no** están en B	{1,2}	El círculo A menos la zona de traslape
Complemento	A^c	Elementos de U que **no** están en A (A^c = U \ A)	{5,6,7,8}	Todo el rectángulo (U) EXCEPTO el círculo A

Diagrama de Venn (U es el rectángulo, A y B los círculos):




```
Solo A (A\B) = {1,2}   Solo B (B\A) = {5,6}
Traslape (A∩B) = {3,4}  Fuera de ambos = {7,8}
```

Propiedades que suelen preguntar:

Propiedad	Fórmula
Conmutativa	A ∪ B = B ∪ A · A ∩ B = B ∩ A

Propiedad	Fórmula
Asociativa	$(A \cup B) \cup C = A \cup (B \cup C)$
Distributiva	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Leyes de De Morgan	$(A \cup B)^c = A^c \cap B^c \cdot (A \cap B)^c = A^c \cup B^c$
Diferencia con complemento	$A \setminus B = A \cap B^c$
Doble complemento	$(A^c)^c = A$
Cardinalidad de la unión	$\|A \cup B\| = \|A\| + \|B\| - \|A \cap B\|$

Comprobación de De Morgan con el ejemplo: $(A \cup B)^c = \{1..6\}^c = \{7,8\}$; $A^c \cap B^c = \{5,6,7,8\} \cap \{1,2,7,8\} = \{7,8\}$. ✓

C.2 Implementación con vectores (arreglos)

Se guardan los elementos en una lista de Python **garantizando que no haya repetidos**. No se usa la clase `set`; todas las operaciones se hacen con ciclos manuales.

```
class ConjuntoVector:
    '''Conjunto implementado con una lista (vector) de Python, sin usar
set.'''

    def __init__(self, elementos=None):
        self.elementos = []                 # el vector que guarda los datos
        if elementos is not None:
            for e in elementos:
                self.agregar(e)             # agregar ya filtra repetidos

    def contiene(self, x):
        '''Búsqueda lineal manual. O(n).'''
        for e in self.elementos:
            if e == x:
                return True
        return False

    def agregar(self, x):
        '''Solo inserta si no existe: así se garantiza la unicidad.'''
        if not self.contiene(x):
            self.elementos.append(x)

    def union(self, otro):
        '''A ∪ B: todo lo de A y todo lo de B (agregar descarta
repetidos).'''
```

```
        resultado = ConjuntoVector()
        for e in self.elementos:
            resultado.agregar(e)
        for e in otro.elementos:
            resultado.agregar(e)
        return resultado

    def interseccion(self, otro):
        '''A ∩ B: elementos de A que también están en B.'''
        resultado = ConjuntoVector()
        for e in self.elementos:
            if otro.contiene(e):
                resultado.agregar(e)
        return resultado

    def diferencia(self, otro):
        '''A \ B: elementos de A que NO están en B.'''
        resultado = ConjuntoVector()
        for e in self.elementos:
            if not otro.contiene(e):
                resultado.agregar(e)
        return resultado

    def complemento(self, universo):
        '''Aᶜ = U \ A. Requiere conocer el universo.'''
        return universo.diferencia(self)

    def __str__(self):
        return '{' + ', '.join(str(e) for e in self.elementos) + '}'


U = ConjuntoVector([1, 2, 3, 4, 5, 6, 7, 8])
A = ConjuntoVector([1, 2, 3, 4, 4, 2])      # los repetidos se descartan
B = ConjuntoVector([3, 4, 5, 6])

print('A      =', A)                        # {1, 2, 3, 4}
print('A ∪ B  =', A.union(B))               # {1, 2, 3, 4, 5, 6}
print('A ∩ B  =', A.interseccion(B))        # {3, 4}
print('A \\ B  =', A.diferencia(B))          # {1, 2}
print('Aᶜ     =', A.complemento(U))         # {5, 6, 7, 8}
```

Complejidad (n = |A|, m = |B|): `contiene` O(n); `agregar` O(n); `union` O(n² + n·m) en el peor caso, simplificable a O((n+m)²); `interseccion` y `diferencia` O(n·m). Es lento, pero didáctico. (Con la clase `set` real, las operaciones son O(1) promedio por elemento gracias al hash; por eso el enunciado prohíbe usarla.)

C.3 Implementación con listas enlazadas

Mismo comportamiento, pero con nodos enlazados. Se guarda `cabeza` y `cola` para insertar al final en O(1) (la unicidad se verifica antes con `contiene`).

```
class NodoConjunto:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None


class ConjuntoLista:
    '''Conjunto implementado con una lista simplemente enlazada (sin
duplicados).'''

    def __init__(self, elementos=None):
        self.cabeza = None
        self.cola = None
        if elementos is not None:
            for e in elementos:
                self.agregar(e)

    def contiene(self, x):
        actual = self.cabeza
        while actual is not None:
            if actual.dato == x:
                return True
            actual = actual.siguiente
        return False

    def agregar(self, x):
        '''Inserta al final solo si x no existe. Devuelve True si se
insertó.'''
        if self.contiene(x):                  # garantiza elementos únicos
            return False
        nuevo = NodoConjunto(x)
        if self.cabeza is None:               # lista vacía
            self.cabeza = self.cola = nuevo
        else:
            self.cola.siguiente = nuevo       # enganchar al final
            self.cola = nuevo
        return True

    def union(self, otro):
        '''A ∪ B: recorre A y luego B, agregando todo (los duplicados se
descartan).'''
```

```
        resultado = ConjuntoLista()
        actual = self.cabeza
        while actual is not None:
            resultado.agregar(actual.dato)
            actual = actual.siguiente
        actual = otro.cabeza
        while actual is not None:
            resultado.agregar(actual.dato)
            actual = actual.siguiente
        return resultado

    def interseccion(self, otro):
        '''A ∩ B: nodos de A cuyo dato también está en B.'''
        resultado = ConjuntoLista()
        actual = self.cabeza
        while actual is not None:
            if otro.contiene(actual.dato):
                resultado.agregar(actual.dato)
            actual = actual.siguiente
        return resultado

    def diferencia(self, otro):
        '''A \ B (extra, muy probable en examen).'''
        resultado = ConjuntoLista()
        actual = self.cabeza
        while actual is not None:
            if not otro.contiene(actual.dato):
                resultado.agregar(actual.dato)
            actual = actual.siguiente
        return resultado

    def __str__(self):
        partes = []
        actual = self.cabeza
        while actual is not None:
            partes.append(str(actual.dato))
            actual = actual.siguiente
        return '{' + ', '.join(partes) + '}'


A = ConjuntoLista([1, 2, 3, 4, 2])
B = ConjuntoLista([3, 4, 5, 6])
print('A ∪ B =', A.union(B))            # {1, 2, 3, 4, 5, 6}
print('A ∩ B =', A.interseccion(B))     # {3, 4}
print('A \\ B =', A.diferencia(B))       # {1, 2}
```

Vectores vs. listas enlazadas para conjuntos:

Aspecto	Vector (lista de Python)	Lista enlazada
Acceso por posición	O(1)	O(n)
Búsqueda de un elemento	O(n)	O(n)
Inserción (ya verificada la unicidad)	O(1) al final (amortizado)	O(1) con puntero `cola`
Eliminar un elemento	O(n) (hay que desplazar)	O(n) buscar + O(1) desenlazar
Memoria	Contigua, sin punteros	Un puntero extra por nodo
Operaciones de conjuntos (n, m)	O(n·m)	O(n·m)

Optimización (por si te la piden): si los elementos se mantienen **ordenados**, unión e intersección se hacen con un recorrido tipo *merge* con dos punteros en O(n + m).

Errores comunes en conjuntos: olvidar verificar duplicados en `agregar` (la unión quedaría con repetidos); implementar el complemento sin universo; modificar el conjunto original en lugar de devolver uno nuevo; en la lista enlazada, perder el puntero `cola` al insertar el primer elemento.

Parte 3 — Simulador de examen parcial (banco de 6 ejercicios)

Código de apoyo para los ejercicios 1 y 2 (nodo genérico y utilidades para construir/ver listas):

```
class Nodo:
    def __init__(self, dato, siguiente=None):
        self.dato = dato
        self.siguiente = siguiente

def desde_lista(valores):
    '''Construye una lista enlazada a partir de una lista de Python.'''
    cabeza = None
    for v in reversed(valores):          # se arma de atrás hacia adelante
        cabeza = Nodo(v, cabeza)
    return cabeza

def a_lista(cabeza):
    '''Convierte la lista enlazada a lista de Python (para verificar
resultados).'''
    resultado = []
    while cabeza is not None:
        resultado.append(cabeza.dato)
        cabeza = cabeza.siguiente
    return resultado
```

Ejercicio 1 (Recursividad) — Invertir una lista enlazada

Enunciado: escribe `invertir(cabeza)` que invierta la lista **recursivamente**, re-enlazando los nodos (sin crear nodos nuevos ni copiar datos) y retornando la nueva cabeza. Ejemplo: `1→2→3→4` pasa a `4→3→2→1`.

Solución paso a paso:

1. **Caso base:** lista vacía o de un solo nodo (`cabeza is None or cabeza.siguiente is None`): ya está invertida, se retorna `cabeza`. Ese último nodo será la nueva cabeza.
2. **Confianza recursiva:** `nueva_cabeza = invertir(cabeza.siguiente)` invierte todo el resto. Tras esa llamada, `cabeza.siguiente` sigue apuntando al que antes era el siguiente, que ahora es el **último** de la parte invertida.
3. **Trabajo propio:** hacer que ese nodo apunte hacia mí (`cabeza.siguiente.siguiente = cabeza`) y cortar mi propio enlace (`cabeza.siguiente = None`).
4. **Retorno:** se devuelve siempre `nueva_cabeza`, sin cambios, hasta el nivel superior.

```
def invertir(cabeza):
    # Caso base: vacía o un solo nodo
    if cabeza is None or cabeza.siguiente is None:
        return cabeza
    nueva_cabeza = invertir(cabeza.siguiente)   # invierte el resto
    cabeza.siguiente.siguiente = cabeza         # el que seguía ahora apunta
atrás, a mí
    cabeza.siguiente = None                     # yo soy el final provisional
    return nueva_cabeza

print(a_lista(invertir(desde_lista([1, 2, 3, 4]))))   # [4, 3, 2, 1]
print(a_lista(invertir(desde_lista([])))) # []
print(a_lista(invertir(desde_lista([7])))) # [7]
```

Traza para `1→2→3→4`:

```
invertir(1) → invertir(2) → invertir(3) → invertir(4)  [base: return 4]

En invertir(3): 3.sig(=4).sig = 3 ; 3.sig = None   →  4 → 3 → None
En invertir(2): 2.sig(=3).sig = 2 ; 2.sig = None   →  4 → 3 → 2 → None
En invertir(1): 1.sig(=2).sig = 1 ; 1.sig = None   →  4 → 3 → 2 → 1 → None
Se retorna 4 en todos los niveles.
```

Errores comunes: (a) olvidar `cabeza.siguiente = None` → queda un ciclo `1 ⇄ 2`; (b) olvidar el caso base de lista vacía → `AttributeError` con `None.siguiente`; (c) retornar `cabeza` en lugar de `nueva_cabeza` → la lista "desaparece" salvo un nodo; (d) usar `nueva_cabeza.siguiente = cabeza` (recorre toda la lista cada vez, O(n²)).

Complejidad: O(n) tiempo, O(n) pila.

Ejercicio 2 (Recursividad) — Eliminar duplicados

Enunciado: escribe `eliminar_duplicados(cabeza)` que deje solo la **primera aparición** de cada valor en una lista **no ordenada**, de forma recursiva. Ejemplo: `3→1→3→2→1` pasa a `3→1→2`.

Solución paso a paso:

1. Necesitamos una función auxiliar `eliminar_valor(nodo, valor)` que quite **todas** las apariciones de un valor en una lista (recursiva, retorna la nueva cabeza).
2. `eliminar_duplicados(nodo)`: caso base lista vacía. Paso recursivo: el nodo actual se queda; se eliminan sus copias del resto con `eliminar_valor` y a ese resto ya limpio se le aplica `eliminar_duplicados`.

```
def eliminar_valor(nodo, valor):
    if nodo is None:                              # caso base
        return None
    if nodo.dato == valor:                        # lo saltamos (no lo
enlazamos)
        return eliminar_valor(nodo.siguiente, valor)
    nodo.siguiente = eliminar_valor(nodo.siguiente, valor)   # re-enlazar
    return nodo

def eliminar_duplicados(nodo):
    if nodo is None:                              # caso base
        return None
    resto_limpio = eliminar_valor(nodo.siguiente, nodo.dato)  # quita copias
de mi dato
    nodo.siguiente = eliminar_duplicados(resto_limpio)         # y limpia el
resto
    return nodo

print(a_lista(eliminar_duplicados(desde_lista([3, 1, 3, 2, 1]))))   # [3, 1,
2]
print(a_lista(eliminar_duplicados(desde_lista([5, 5, 5]))))         # [5]
print(a_lista(eliminar_duplicados(desde_lista([1, 2, 3]))))         # [1, 2,
3]
```

Traza para `3→1→3→2→1`:

```
eliminar_duplicados(3)  rest = eliminar_valor(1→3→2→1, 3) = 1→2→1
 └ eliminar_duplicados(1)  rest = eliminar_valor(2→1, 1) = 2
    └ eliminar_duplicados(2)  rest = eliminar_valor(None, 2) = None
       └ eliminar_duplicados(None) → None   (CASO BASE)
    2.sig = None → return 2
 1.sig = 2 → return 1
3.sig = 1 → return 3        Resultado: 3 → 1 → 2
```

Errores comunes: (a) comparar solo con el nodo **adyacente** (`nodo.dato == nodo.siguiente.dato`): solo sirve en listas **ordenadas**; (b) eliminar solo la primera copia con `eliminar_valor` en vez de todas; (c) no re-enlazar (`nodo.siguiente = ...`).

Complejidad: $O(n^2)$ en el peor caso (cada nodo dispara un recorrido del resto), O(n) pila. Si la lista estuviera ordenada, bastaría comparar con el siguiente y sería O(n).

Ejercicio 3 (Regex) — Validar placas de vehículos colombianos

Enunciado: valida placas de **carro** (`ABC123` : 3 letras mayúsculas + 3 dígitos) y de **moto** (`ABC12D` : 3 letras + 2 dígitos + 1 letra). Debe aceptar un espacio o guion opcional entre las letras y el resto (`ABC-123` , `ABC 12D`).

Solución paso a paso:

1. Bloque de letras: `[A-Z]{3}` .
2. Separador opcional: `[- ]?` (guion o espacio; el guion al final de la clase es literal).
3. Dos variantes del final: `\d{3}` (carro) **o** `\d{2}[A-Z]` (moto) → alternación agrupada con `(?:...|...)` .
4. Anclas `^` y `$` (o `fullmatch`) para validar toda la cadena.

```
import re

placa = re.compile(r'^[A-Z]{3}[- ]?(?:\d{3}|\d{2}[A-Z])$')

pruebas = {
    'ABC123': True,   # carro
    'ABC-123': True,  # con guion
    'XYZ12D': True,   # moto
    'ABC 12D': True,  # moto con espacio
    'abc123': False,  # minúsculas
    'AB1234': False,  # solo 2 letras
    'ABC12': False,   # incompleta
    'ABC1234': False, # sobran dígitos
}
for texto, esperado in pruebas.items():
    resultado = bool(placa.fullmatch(texto))
    print(f'{texto:10} -> {resultado}  {"OK" if resultado == esperado else
"ERROR"}')
```

Errores comunes: (a) sin paréntesis la alternación `\d{3}|\d{2}[A-Z]$` rompe todo el patrón (el `|` separa **todo** lo de la izquierda y la derecha); (b) olvidar las anclas, con lo cual `ABC1234` pasaría con `search` ; (c) escribir `[A-z]` (incluye símbolos entre `Z` y `a`); (d) olvidar `r''` .

Ejercicio 4 (Regex) — Extraer y sumar precios en pesos

Enunciado: del texto `'Camisa $45.000, zapatos $ 120.500 y gorra $ 15.000,50. Descuento 10%.'` extrae todos los precios (punto como separador de miles, coma decimal, espacio opcional tras `$`) y calcula el total.

Solución paso a paso:

1. `\$` escapa el signo de pesos; `\s?` permite un espacio opcional.
2. Parte entera: `(?:\d{1,3}(?:\.\d{3})+|\d+)` → o bien grupos de miles (`45.000`) o bien dígitos simples (`1234`).
3. Parte decimal opcional: `(?:,\d{2})?` .
4. Convertir a `float` : quitar `$` y espacios, quitar los puntos (miles) y cambiar la coma por punto.

```
import re

texto = 'Camisa $45.000, zapatos $ 120.500 y gorra $ 15.000,50. Descuento
10%.'
patron_precio = r'\$\s?(?:\d{1,3}(?:\.\d{3})+|\d+)(?:,\d{2})?'

precios = re.findall(patron_precio, texto)
print(precios)    # ['$45.000', '$ 120.500', '$ 15.000,50']

def a_numero(s):
    s = s.replace('$', '').strip()          # '120.500'
    s = s.replace('.', '').replace(',', '.')  # '120500' o '15000.50'
    return float(s)

total = sum(a_numero(p) for p in precios)
print(total)      # 180500.5
```

Errores comunes: (a) olvidar escapar `$` (en regex significa fin de línea); (b) escribir `\d+\.\d+` pensando en formato inglés y capturar `45.000` como decimal; (c) convertir con `float('45.000')` directamente (da 45.0, no 45000); (d) usar `findall` con grupos de captura: devolvería solo el contenido de los grupos, por eso los grupos son **sin captura** `(?:...)` .

Ejercicio 5 (Conjuntos con vectores) — Conjunto como vector característico

Enunciado: implementa un conjunto sobre el universo `{0,…,N-1}` usando un **vector de booleanos** (`v[i]` es `True` si `i` pertenece). Implementa unión, intersección, diferencia y complemento, y calcula con N = 10: A = {1,2,3,4}, B = {3,4,5,6}.

Solución paso a paso: (1) El índice **es** el elemento: pertenencia en O(1). (2) Cada operación es un recorrido de las N posiciones aplicando un operador lógico: unión = `or` , intersección = `and` , diferencia = `a and not b` , complemento = `not a` . (3) Ambos conjuntos deben compartir el mismo `N` .

```
class ConjuntoBits:
    def __init__(self, n, elementos=()):
        self.n = n                          # tamaño del universo {0..n-1}
        self.v = [False] * n                # v[i] == True  <=>  i está en el
conjunto
        for e in elementos:
            self.v[e] = True                # marcar presencia (repetidos no
afectan)

    def _operar(self, otro, f):
        if self.n != otro.n:
            raise ValueError('Los conjuntos deben tener el mismo universo')
        r = ConjuntoBits(self.n)
        for i in range(self.n):             # recorrido posición a posición
            r.v[i] = f(self.v[i], otro.v[i])
        return r

    def union(self, otro):        return self._operar(otro, lambda a, b: a or
b)
    def interseccion(self, otro): return self._operar(otro, lambda a, b: a
and b)
    def diferencia(self, otro):   return self._operar(otro, lambda a, b: a
and not b)

    def complemento(self):
        r = ConjuntoBits(self.n)
        for i in range(self.n):
            r.v[i] = not self.v[i]
        return r

    def elementos(self):
        return [i for i in range(self.n) if self.v[i]]


A = ConjuntoBits(10, [1, 2, 3, 4])
B = ConjuntoBits(10, [3, 4, 5, 6])
print(A.union(B).elementos())          # [1, 2, 3, 4, 5, 6]
print(A.interseccion(B).elementos())   # [3, 4]
print(A.diferencia(B).elementos())     # [1, 2]
print(A.complemento().elementos())     # [0, 5, 6, 7, 8, 9]
```

Errores comunes: (a) `IndexError` al insertar un elemento ≥ N; (b) operar conjuntos de universos distintos; (c) olvidar que el complemento cambia según N; (d) devolver el mismo objeto modificado en lugar de uno nuevo.

Complejidad: todas las operaciones O(N); pertenencia y agregar O(1). Ventaja: muy rápido; desventaja: la memoria depende del tamaño del universo, no de la cantidad de elementos.

Ejercicio 6 (Conjuntos con listas enlazadas) — Subconjunto y diferencia simétrica

Enunciado: a la clase `ConjuntoLista` de la sección C.3, agrégale `es_subconjunto(otro)` (A ⊆ B) y `diferencia_simetrica(otro)` (A △ B = elementos que están en exactamente uno de los dos).

Solución paso a paso: (1) A ⊆ B si **todo** nodo de A está en B: se recorre A y se retorna `False` apenas un dato no esté en B; al terminar, `True` (el vacío es subconjunto de todo). (2) A △ B = (A \ B) ∪ (B \ A): se reutilizan `diferencia` y `union` ya construidas.

```
# Métodos para agregar DENTRO de la clase ConjuntoLista:

    def es_subconjunto(self, otro):
        '''True si todos los elementos de self están en otro (self ⊆
otro).'''
        actual = self.cabeza
        while actual is not None:
            if not otro.contiene(actual.dato):   # un elemento fuera -> no es
subconjunto
                return False
            actual = actual.siguiente
        return True                              # incluye el caso del
conjunto vacío

    def diferencia_simetrica(self, otro):
        '''A △ B = (A \ B) ∪ (B \ A).'''
        return self.diferencia(otro).union(otro.diferencia(self))


# Pruebas
A = ConjuntoLista([1, 2])
B = ConjuntoLista([1, 2, 3])
print(A.es_subconjunto(B))      # True
print(B.es_subconjunto(A))      # False
print(ConjuntoLista().es_subconjunto(A))   # True (vacío ⊆ cualquiera)

C = ConjuntoLista([1, 2, 3, 4])
D = ConjuntoLista([3, 4, 5, 6])
print(C.diferencia_simetrica(D))   # {1, 2, 5, 6}
```

Errores comunes: (a) invertir la dirección (preguntar si los de `otro` están en `self`); (b) comparar dos conjuntos por igualdad de listas **en orden** (`{1,2}` y `{2,1}` son el mismo conjunto: dos conjuntos son iguales si A ⊆ B **y** B ⊆ A); (c) olvidar que la diferencia simétrica no es la unión menos la intersección calculada con un solo recorrido sin verificar duplicados.

Complejidad: `es_subconjunto` O(n·m); `diferencia_simetrica` O(n·m + m·n) = O(n·m).

Checklist final de repaso

Tema	Debes poder...
Listas	Dibujar los enlaces antes/después de insertar y eliminar en simple, circular y doble; manejar los casos de lista vacía, un nodo, cabeza y final
Recursividad	Identificar caso base y paso recursivo; decidir si el helper debe **retornar el nodo** (modifica) o **un valor** (consulta); dibujar la pila de IDA y REGRESO
Complejidad	Escribir la recurrencia y resolverla (T(n)=T(n−1)+1 → O(n); T(n)=T(n/2)+1 → O(log n); fib → $O(2^n)$)
Regex	Anclar con `^` / `$` o `fullmatch`; escapar `.`, `$`, `+`, `(`; usar grupos sin captura; explicar un patrón pieza por pieza; simplificar con identidades
Conjuntos	Calcular a mano ∪, ∩, \, ^{c} y dibujar el Venn; implementar unicidad en `agregar`; saber que el complemento necesita el universo

Estrategia para el parcial: (1) escribe primero el caso base y verifica con una lista de 0 y 1 elementos; (2) haz la traza con una lista de 3 nodos antes de entregar; (3) comenta el invariante ("el helper retorna la cabeza de la sublista ya procesada"); (4) revisa los `None` antes de acceder a `.siguiente`.